CSA0619-Design and Analysis of Algorithm

CO1_ASSESSMENT TOOL 1 _Application-Based Question

06. Banking Transaction Processing System:

main.py

```python
import math

a = 4
b = 2

print("Banking Transaction Processing System")
print("Recurrence Relation: T(n) = 4T(n/2) + n^2")

log_value = math.log(a, b)

print("\na =", a)
print("b =", b)
print("f(n) = n^2")
print("log_b(a) =", log_value)

if log_value == 2:
    print("\nMaster Theorem Case: Case 2")
    print("Time Complexity = Θ(n^2 log n)")
```

Output

```
Banking Transaction Processing System
Recurrence Relation: T(n) = 4T(n/2) + n^2

a = 4
b = 2
f(n) = n^2
log_b(a) = 2.0

Master Theorem Case: Case 2
Time Complexity = Θ(n^2 log n)

=== Code Execution Successful ===
```

Analysis:

The algorithm performs efficiently for moderate transaction volumes. However, as the number of banking transactions increases, the execution time grows proportionally to , making it more expensive for very large datasets.

Asymptotic Interpretation:

The asymptotic notation **Θ(n² log n)** indicates that the running time grows proportionally to for large input sizes.

Report Analysis:

Case	Time Complexity
Best Case	Θ(n² log n)
Average Case	Θ(n² log n)
Worst Case	Θ(n² log n)

40. Application: Genome Sequencing Analysis:

main.py

Share Run

```
1  import math
2
3  # Parameters
4  a = 3
5  b = 2
6
7  print("Genome Sequencing Analysis")
8  print("Recurrence Relation: T(n) = 3T(n/2) + n")
9
10 # Calculate log_b(a)
11 log_value = math.log(a, b)
12
13 print("\na =", a)
14 print("b =", b)
15 print("f(n) = n")
16 print("log_b(a) =", round(log_value, 3))
17
18 # Apply Master Theorem
19 print("\nMaster Theorem Case: Case 1")
20 print("Time Complexity = 0(n^1.585)")
```

Output

```
Genome Sequencing Analysis
Recurrence Relation: T(n) = 3T(n/2) + n

a = 3
b = 2
f(n) = n
log_b(a) = 1.585

Master Theorem Case: Case 1
Time Complexity = 0(n^1.585)

=== Code Execution Successful ===
```

Analysis:

The algorithm grows slower than quadratic time. As the size of biological datasets increases, the algorithm remains more scalable and efficient than algorithms with O(n²) complexity, making it suitable for large genome sequencing tasks.

Asymptotic Interpretation:

The asymptotic notation Θ(n^1.585) indicates that the running time grows approximately proportional to n^1.585 for large input sizes.

Report Analysis:

Case	Time Complexity
Best Case	Θ(n^1.585)
Average Case	Θ(n^1.585)
Worst Case	Θ(n^1.585)

Space Complexity: O(log n)